

OOP (Object Oriented Programming)

OOP in Python

Everything is an object.

- The int object 10 is instantiated.
- The variable i is referencing the object.

```
i = 10 # Python
```

This is Java equivalent.

```
Integer i = new Integer(10);
```

| `__init__` as a Constructor

- Object creation in Python is two steps:
`__new__` creates the instance,
`__init__(self)` initializes it.
- Custom classes almost always only override `__init__`, so it's commonly called "the constructor."

```
class ...:
    def __init__(self):
        self.speed = 0
        self.altitude = 0
        self.rollAngle = 0
        self.pitchAngle = 0
```

- We can use parameters and default values.

```
# when no argument is given,  
# default values are assigned  
def __init__(self, speed = 0,  
              altitude = 0,  
              rollAngle = 0):  
    self.speed = speed  
    self.altitude = altitude  
    self.rollAngle = rollAngle
```

Instantiation

- No **new** needed; everything in Python is an object.
- Objects are created, and references point to them.

```
# Airplane.__init__(self) is invoked.  
a = Airplane()
```

Overriding Python class methods

- By default, printing a user-defined object shows a representation with its class name and identity, not just a bare memory address.
- Override `__str__` to print a custom, readable string instead.

```
a = Airplane(speed = 100)
# <__main__.Airplane object at 0x106f13220>
print(a)

class Airplane(object):
    ...
    def __str__(self):
        return f"Speed: {self.speed} ..."

b = Airplane(speed = 100)
print(a) # Speed: 100 ...
```

Overriding `__eq__`

- When we override the `__eq__()` method, we can change how the `==` operator works.

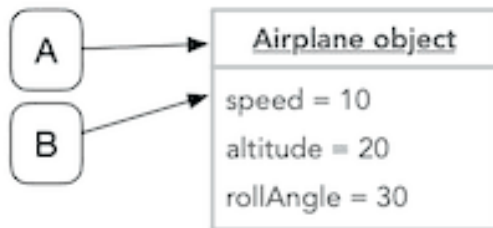
```
a = Airplane(speed = 100)
b = Airplane(speed = 100)
# False, as they are different object
print(a == b)
```

We need to return true when the speed value is the same.

```
class Airplane(object):  
    ...  
    def __eq__(self, other): # overriding ==  
        return self.speed == other.speed  
  
a = Airplane(speed = 100)  
b = Airplane(speed = 100)  
  
# True because the speeds are the same  
print(a == b)
```

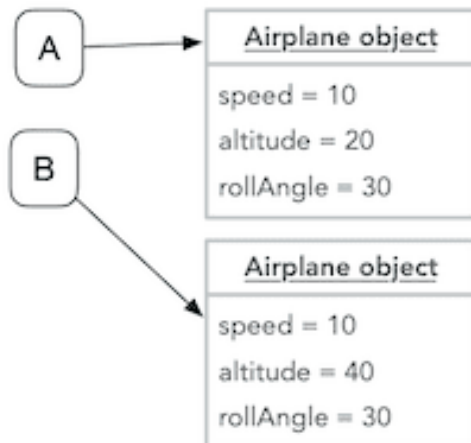
Shallow and Deep Copy

- In OOP, the `=` (assignment) operator copies the reference, not the object — both names end up pointing at the same object.



```
1 a = Airplane(speed = 100)
2 b = a
3 print(a == b) # True as same reference
4 b.speed = 500
5 print(a.speed) # 500
```

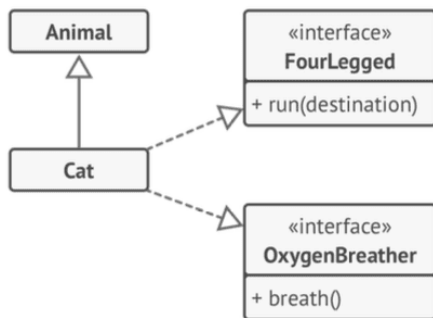
- To make a new, independent object, use `copy.deepcopy()`, which also copies any nested mutable objects. Use `copy.copy()` for a shallow copy (new outer object, still-shared nested objects). A few objects customize this via `__deepcopy__`.



```
1 from copy import deepcopy
2 ...
3 a = Airplane(speed = 100)
4 b = deepcopy(Airplane(speed = 100))
5 print(a == b) # False
6 b.speed = 500
7 print(a.speed) # 100
```

Python and Interface

- Python does not have an interface, but we can mimic an interface with an empty method.



```
1 class Animal(object): # Normal class
2     def makeSound(): print("")
3 class FourLegged(object): # Interface
4     def run(self, destination): pass
5 class OxygenBreather(object): # interface
6     def breathe(self): pass
7
8 # To python extends, and implements are the same
9 class Cat(Animal, FourLegged, OxygenBreather):
10     def run(self, destination):
11         print(f"I run to {destination}")
12     def breathe(self): print("I breathe")
13
14 c = Cat()
15 c.run("NKU")
16 c.breathe()
```

Python Duck Typing

- Duck typing means Python cares about an object's behavior, not its type.
- If an object has the required methods/attributes, it can be used regardless of its class.
- "If it walks like a duck and quacks like a duck, it's a duck."

- Both have the `__len__` method, so the `len` function invokes the method without knowing anything about the object.

```
# len is used for both data structures.  
a = []; len(a) # a.__len__()  
b = (); len(b) # b.__len__()
```

- We have the Duck and Tiger class that has the quack() method.
- The method is invoked in the same way.

```
class Duck(): def quack(self): print("Quack")  
class Tiger(): def quack(self): print("Wow")  
def quack_checker(q): q.quack()
```

```
quack_checker(Duck())  
quack_checker(Tiger())
```

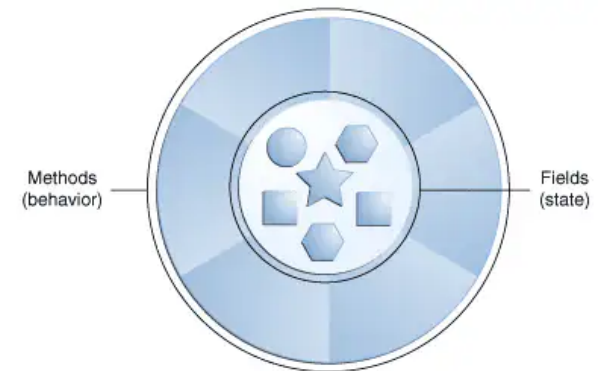
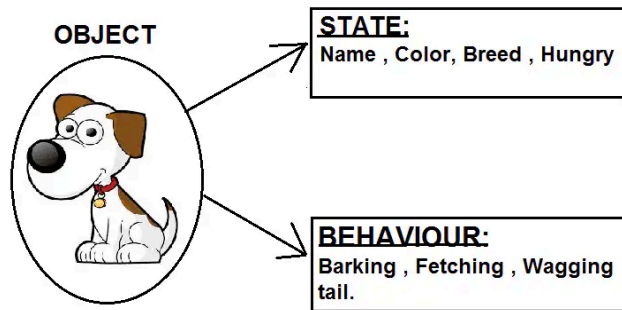
Python: Modeling with OOP

Python is an OOP language that can model the world effectively.

- Abstraction
- Polymorphism
- Inheritance
- Encapsulation

Abstraction

- Software modeling represents the real world.
- This is called abstraction.
- Abstraction captures an object's states (nouns) and behaviors (verbs).



- A Dog object has states and behaviors.
- The states/behaviors are called states/methods.

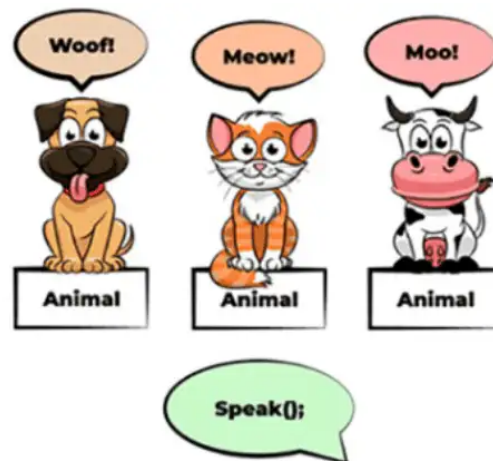
| Polymorphism

- One message, different actions — this is polymorphism. The same method call behaves differently depending on the calling object's class.
- Separately, Python provides `isinstance()`, a type-check function: it checks whether an object is an instance of a class and returns `True` or `False`. It's a useful tool nearby, but it doesn't check for polymorphism itself.

```
class Dog:  
    pass
```

```
class Cat:  
    pass
```

```
animal = Dog()  
print(isinstance(animal, Dog)) # True  
print(isinstance(animal, Cat)) # False
```



Inheritance

- We can model the real world with hierarchical general and specific concepts.
- This is called inheritance: superclasses are general, subclasses are specific, and creating one is called subclassing.

Super Class



Animals

Child Class



Amphibians



Reptiles



Mammals



Birds

Encapsulation

- In OOP, encapsulation means keeping fields (state) private and using public methods to access them.

